ros2kws — *тележка quasi*cart + xArm6 с присоской и камерой (ROS 2 Kilted, Gazebo Sim)

Рабочее пространство ROS 2 с симуляцией манипулятора **xArm6** с вакуумным гриппером и камерой **RealSense D435i** на фланце, установленного на тележку **quasi_cart** (пакеты `r2_robot`). Камера гриппера даёт изображение и облако точек, а `grasp_analyzer` по нему генерирует варианты захвата цилиндрического блока присоской. Распознавание объектов планируется добавить позже.

Сценарии запуска:

#	Что	Команда
B1	Тележка + рука: Gazebo + RViz с MoveIt + grasp_analyzer	`ros2 launch cart_xarm_bringup cart_xarm6_gz.launch.py`
B2	Gazebo + обычный RViz, без MoveIt	`... cart_xarm6_gz.launch.py use_moveit:=false`
B3	Только Gazebo (без RViz и MoveIt)	`... cart_xarm6_gz.launch.py use_moveit:=false use_rviz:=false`
B4	Только RViz, без Gazebo (модель + ползунки суставов)	`ros2 launch cart_xarm_bringup cart_xarm6_rviz.launch.py`
A	Рука отдельно на столе: Gazebo + MoveIt + RViz	`ros2 launch xarm_gz_bringup xarm6_gz.launch.py add_vacuum_gripper:=true`
A2	Рука отдельно, только RViz + MoveIt (фейковое железо)	`ros2 launch xarm_moveit_config xarm6_moveit_fake.launch.py add_vacuum_gripper:=true`

---

1. Требования

Проверенная конфигурация:

- **Ubuntu 24.04** (x86_64)
- **ROS 2 Kilted Kaiju**
- **Gazebo Sim 9** (ставится вместе с `ros-kilted-ros-gz`, отдельно ставить не нужно)
- видеокарта с OpenGL 3.3+ (камера в симуляции рендерится через ogre2)

Другие дистрибутивы ROS (Humble/Jazzy) не проверялись: launch-файлы и плагины настроены под `gz_ros2_control` и gz sim 9 из Kilted.

2. Установка ROS 2 Kilted

Если ROS 2 уже стоит, переходите к разделу 3.

```
# локаль UTF-8
  sudo apt update && sudo apt install -y locales
  sudo locale-gen en_US en_US.UTF-8
  sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
  export LANG=en_US.UTF-8

  # репозиторий ROS 2
  sudo apt install -y software-properties-common curl
  sudo add-apt-repository universe
  export ROS_APT_SOURCE_VERSION=$(curl -s https://api.github.com/repos/ros-infrastructu
  re/ros-apt-source/releases/latest | grep -F "tag_name" | awk -F\" '{print $4}')
  curl -L -o /tmp/ros2-apt-source.deb "https://github.com/ros-infrastructure/ros-apt-so
  urce/releases/download/${ROS_APT_SOURCE_VERSION}/ros2-apt-source_${ROS_APT_SOURCE_VER
  SION}.$(. /etc/os-release && echo $VERSION_CODENAME)_all.deb"
  sudo dpkg -i /tmp/ros2-apt-source.deb

  sudo apt update
  sudo apt install -y ros-kilted-desktop ros-dev-tools
```

Чтобы не вводить `source` в каждом терминале:

```
echo "source /opt/ros/kilted/setup.bash" >> ~/.bashrc
```

3. Зависимости проекта

```
sudo apt install -y \
    python3-colcon-common-extensions python3-numpy python3-opencv \
    ros-kilted-xacro \
    ros-kilted-ros-gz \
    ros-kilted-gz-ros2-control \
    ros-kilted-ros2-control ros-kilted-ros2-controllers ros-kilted-ros2controlcli \
    ros-kilted-moveit \
    ros-kilted-joint-state-publisher ros-kilted-joint-state-publisher-gui \
    ros-kilted-robot-state-publisher \
    ros-kilted-cv-bridge ros-kilted-image-transport \
    ros-kilted-tf2-ros ros-kilted-tf2-tools \
    ros-kilted-rviz2 \
    ros-kilted-teleop-twist-keyboard
```

Если что-то забыто, `rosdep` доставит недостающее (запускать после клонирования, раздел 4):

```
sudo rosdep init        # один раз на машину
  rosdep update
  cd ~/ros2k_ws && rosdep install --from-paths src --ignore-src -r -y
```

4. Клонирование

Корень репозитория — это само рабочее пространство (внутри лежит `src/`):

```
git clone https://github.com/TolyaDenisenko/ros2k_ws.git ~/ros2k_ws
  cd ~/ros2k_ws
```

Папка `src/object_recognition_msgs` после клонирования пустая: это ссылка на сабмодуль без `.gitmodules`. Она ни на что не влияет, потому что пакет, которому она нужна (`d435i_xarm_setup`), исключён из сборки.

5. Сборка

Собирать нужно **из** `~/ros2k_ws`, а не из `src/`:

```
cd ~/ros2k_ws
  source /opt/ros/kilted/setup.bash
  colcon build --symlink-install
  source install/setup.bash
```

`--symlink-install` нужен, чтобы правки URDF/xacro, launch-файлов, миров и python-скриптов подхватывались без пересборки (кроме новых файлов: их нужно установить, т.е. пересобрать пакет).
Чтобы окружение подхватывалось в каждом новом терминале:

```
echo "source ~/ros2k_ws/install/setup.bash" >> ~/.bashrc
```

Пакеты, исключённые из сборки

В `src/xarm_ros2` есть пакеты под Gazebo Classic и зависимости, которых нет в Kilted. В каждом лежит `COLCON_IGNORE`, colcon их пропускает:

Пакет	Почему исключён
`thirdparty/realsense_gazebo_plugin`	плагин для Gazebo Classic (`gazebo_ros`); камера в gz sim сделана нативным SDF-сенсором
`xarm_gazebo`	Gazebo Classic; вместо него `xarm_gz_bringup`
`xarm_vision/d435i_xarm_setup`	нужен `find_object_2d` / `object_recognition_msgs`, нужен только для реальной камеры
`xarm_planner`	метапакет `moveit`, недоступный в Kilted; планирование даёт `xarm_moveit_config`
`xarm_moveit_servo`	несовместим с API `moveit_servo` текущей версии

Проверка сборки

```
ros2 pkg list | grep -E "xarm|cart"
  # ожидаем среди прочего: cart_xarm_bringup, quasi_cart_description,
  # xarm_description, xarm_controller, xarm_moveit_config, xarm_gz_bringup, xarm_graspi
  ng
```

Если после `git pull` сборка ведёт себя странно, сделайте чистую сборку:

```
cd ~/ros2k_ws && rm -rf build install log && colcon build --symlink-install
```

---

6. Сценарий B — рука на тележке

B1. Gazebo + RViz с MoveIt (основной запуск)

```
ros2 launch cart_xarm_bringup cart_xarm6_gz.launch.py
```

Открываются Gazebo и одно окно RViz (`cart_xarm_moveit.rviz`). В нём модель тележки с рукой в той же позе, что и в Gazebo, панель **MotionPlanning**, маркеры захвата и изображение с камеры гриппера.

Чтобы подвигать руку: перетащите интерактивный маркер на конце руки (или выберите Goal State во вкладке Planning) и нажмите **Plan & Execute**. Траектория уходит в `xarm6_traj_controller` в Gazebo, и рука двигается одновременно в Gazebo и RViz.

- Модель MoveIt содержит **только руку**, тележки в ней нет: столкновения руки с тележкой MoveIt не проверяет.
- Кадр `world` MoveIt совпадает с `link_base` (основание руки), Fixed Frame RViz — `base_link` тележки.
- В логе `move_group` будут строки `Could not find the planner configuration 'RRTConnect' on the param server`. Это особенность upstream-конфига xArm, на работу не влияет: планирование идёт через OMPL по умолчанию.

Что поднимается (файл `src/r2_robot/cart_xarm_bringup/launch/cart_xarm6_gz.launch.py`):

t, с	Что
0	Gazebo с миром `cart_table_offline.world` (стол + цилиндрический блок, работает без интернета)
0	`robot_state_publisher` с URDF `cart_with_arm.urdf.xacro`, `joint_state_merger`, `gz_bridge`, RViz (если `use_moveit:=false`)
4	спавн модели `cart_xarm6` (рука сразу в начальной позе, см. раздел 8)
7	контроллеры `joint_state_broadcaster` и `xarm6_traj_controller`
10	`grasp_analyzer`
12	MoveIt: `move_group` + RViz с MotionPlanning (если `use_moveit:=true`, по умолчанию)

Аргументы launch-файла:

Аргумент	По умолчанию	Назначение
`use_moveit`	`true`	MoveIt и RViz `cart_xarm_moveit.rviz` (тележка + MotionPlanning + маркеры + камера)
`use_rviz`	`true`	обычный RViz `cart_xarm.rviz`; открывается только при `use_moveit:=false`
`arm_initial_positions`	`1.5708 -0.9 -1.3 0 1.0 0`	начальные углы joint1..joint6, рад
`world`	`.../worlds/cart_table_offline.world`	мир; `cart_table.world` требует интернет (Fuel)
`spawn_x`, `spawn_y`, `spawn_z`, `spawn_yaw`	`0`, `0`, `0.07`, `0`	поза спавна тележки (стол при x≈+1.7)
`robot_model`	`c2`	модель тележки из `quasi_cart_description/urdf`

B2. Gazebo + обычный RViz, без MoveIt

```
ros2 launch cart_xarm_bringup cart_xarm6_gz.launch.py use_moveit:=false
```

Запускается легче. Рукой можно управлять напрямую через контроллер (раздел 10).

B3. Только Gazebo

```
ros2 launch cart_xarm_bringup cart_xarm6_gz.launch.py use_moveit:=false use_rviz:=false
```

RViz потом можно открыть отдельно в другом терминале:

```
ros2 run rviz2 rviz2 -d ~/ros2k_ws/install/cart_xarm_bringup/share/cart_xarm_bringup/rviz/cart_xarm.rviz \
  --ros-args -p use_sim_time:=true
```

B4. Только RViz (без Gazebo)

```
ros2 launch cart_xarm_bringup cart_xarm6_rviz.launch.py
```

Открываются RViz с моделью тележки и руки и окно **Joint State Publisher** с ползунками всех суставов (рука, колёса). Ползунки стартуют в той же боковой позе, что и симуляция. Подходит, чтобы проверить URDF, крепление руки и TF без запуска физики. `use_gui:=false` — без окна ползунков.

Что смотреть в RViz

Конфиги `cart_xarm.rviz` и `cart_xarm_moveit.rviz` (Fixed Frame `base_link`):

- **RobotModel** — тележка и рука по `/robot_description`;
- **TF** — все кадры. Ни один не должен быть красным, включая `drive_wheel_*_link`;
- **MarkerArray** `/xarm_grasping/grasp_markers` — варианты захвата;
- **Image** `/camera/image_display` — картинка с камеры гриппера.

---

7. Сценарий A — рука отдельно на столе

A. Gazebo + MoveIt

```
ros2 launch xarm_gz_bringup xarm6_gz.launch.py add_vacuum_gripper:=true
```

- Мир `table_gz.world` берёт пол, свет и стол из Gazebo Fuel, поэтому **при первом запуске нужен интернет** (дальше модели берутся из кэша `~/.gz/fuel`).
- Гравитация в этом мире выключена (`<gravity>0 0 0</gravity>`), так задумано в upstream-примере.
- Рука спавнится на столешнице (`z=1.015`) в начальной позе `0 -0.6 -0.7 0 1.3 0`: инструмент смотрит вниз, присоска в 0.29 м над столом. Поменять позу можно через `arm_initial_positions:="..."`.
- MoveIt и его RViz стартуют через 12 с после Gazebo.

Анализатор захвата в этом сценарии запускается отдельно (камеры в нём нет, поэтому он работает по априорной позе блока):

```
ros2 run xarm_grasping grasp_analyzer.py --ros-args -p use_sim_time:=true
```

A2. Только RViz + MoveIt (без Gazebo)

```
ros2 launch xarm_moveit_config xarm6_moveit_fake.launch.py add_vacuum_gripper:=true
```

Здесь вместо Gazebo используется «фейковое» железо ros2_control: Plan & Execute двигает руку только в RViz. Удобно, чтобы проверить планирование без физики.

---

8. Начальная поза руки

Начальная поза задаётся параметром `initial_value` в ros2_control (`xarm6.ros2_control.xacro`, xacro-аргумент `xarm_initial_positions`). `gz_ros2_control` выставляет суставы в эту позу в момент спавна, а `xarm6_traj_controller` при активации её удерживает: рука не стартует из нуля и никуда не переезжает.

Сценарий	Поза по умолчанию	Почему
B (тележка)	`1.5708 -0.9 -1.3 0 1.0 0`	рука развёрнута вбок (+Y тележки, стол по +X), сложена над платформой; камера смотрит в сторону и на ~20° вниз; локоть и фланец в пределах габарита тележки
A (стол)	`0 -0.6 -0.7 0 1.3 0`	в нулевой позе кончик присоски (фланец 0.112 м + присоска 0.126 м) уходил под столешницу, контакт сбивал суставы, MoveIt прерывал траектории с `PATH_TOLERANCE_VIOLATED`

Своя поза:

```
ros2 launch cart_xarm_bringup cart_xarm6_gz.launch.py arm_initial_positions:="0 0 0 0 0
  0"
  ros2 launch xarm_gz_bringup xarm6_gz.launch.py add_vacuum_gripper:=true arm_initial_p
  ositions:="0 -0.5 -0.5 0 1.0 0"
```

Для реального робота (`uf_robot_hardware/UFRobotSystemHardware`) параметр игнорируется.

---

9. Время симуляции (`use_sim_time`)

Отдельная команда синхронизации времени больше не нужна. Раньше приходилось после запуска вручную выполнять что-то вроде

```
for n in $(ros2 node list); do ros2 param set $n use_sim_time true; done
```

иначе рука не двигалась из MoveIt. Причина была в `xarm_moveit_config/launch/_robot_moveit_common.launch.py`: он передаёт `move_group`, RViz и static TF параметр `{'use_sim_time': <аргумент>}` **явно**, со значением по умолчанию `False`. Явный параметр ноды перекрывает глобальный `SetParameter('use_sim_time', True)` из нашего launch-файла. `move_group` жил по системному времени, а состояния суставов из Gazebo шли с временем симуляции, поэтому MoveIt считал их устаревшими и не исполнял траектории.

Сейчас оба launch-файла (`cart_xarm6_gz.launch.py` и `xarm6_gz.launch.py`) передают в MoveIt `use_sim_time: 'true'` напрямую, и фоновый «фиксер» удалён.

Проверка:

```
ros2 param get /move_group use_sim_time      # Boolean value is: True
  ros2 topic hz /clock                         # время идёт из Gazebo
```

Если какая-то нода, запущенная вручную, ведёт себя странно, запускайте её с `--ros-args -p use_sim_time:=true`.

---

10. Управление

Рука через контроллер (без MoveIt)

```
ros2 topic pub --once /xarm6_traj_controller/joint_trajectory trajectory_msgs/msg/Joint
  Trajectory \
    "{joint_names: [joint1, joint2, joint3, joint4, joint5, joint6],
      points: [{positions: [1.5708, -0.6, -1.0, 0.0, 1.0, 0.0], time_from_start: {sec:
  3}}]}"
```

Рука через MoveIt из терминала (запуск по умолчанию, с MoveIt)

```
ros2 action send_goal /move_action moveit_msgs/action/MoveGroup "{request: {group_name:
  xarm6,
    max_velocity_scaling_factor: 0.5, max_acceleration_scaling_factor: 0.5, allowed_pla
  nning_time: 5.0,
    goal_constraints: [{joint_constraints: [
      {joint_name: joint1, position: 1.0, tolerance_above: 0.01, tolerance_below: 0.01,
  weight: 1.0},
      {joint_name: joint2, position: -0.9, tolerance_above: 0.01, tolerance_below: 0.0
  1, weight: 1.0},
      {joint_name: joint3, position: -1.3, tolerance_above: 0.01, tolerance_below: 0.0
  1, weight: 1.0},
      {joint_name: joint4, position: 0.0, tolerance_above: 0.01, tolerance_below: 0.01,
  weight: 1.0},
      {joint_name: joint5, position: 1.0, tolerance_above: 0.01, tolerance_below: 0.01,
  weight: 1.0},
      {joint_name: joint6, position: 0.0, tolerance_above: 0.01, tolerance_below: 0.01,
  weight: 1.0}]}]},
    planning_options: {plan_only: false}}"
  # ожидаем: Goal finished with status: SUCCEEDED
```

Тележка

```
ros2 topic pub -r 10 /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.2}}"   # Ctrl+C — стоп
ros2 topic echo /odom --field pose.pose.position
# или с клавиатуры:
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

Тележку двигает плагин Gazebo `VelocityControl`: он перемещает корпус целиком, поэтому колёса при езде не вращаются. TF `odom → base_footprint` в ROS не публикуется (RViz работает в кадре `base_link`).

Присоска

Физического захвата присоской в симуляции пока **нет**: гриппер — модель со звеньями, без плагина вакуума. Сервиса включения присоски в Gazebo нет.

---

11. Анализатор захватов (`xarm_grasping`)

`grasp_analyzer.py` в сценарии B стартует сам. Он:

- подписывается на `/camera/depth/points` и `/camera/color/image_raw`;
- каждые 2 с ищет столешницу и объект в облаке точек;
- генерирует варианты захвата присоской (TOP / RIM / SIDE) и публикует:
  - `/xarm_grasping/grasp_candidates` — JSON со всеми вариантами,
  - `/xarm_grasping/best_grasp_pose` — лучший вариант (PoseStamped в `link_base`),
  - `/xarm_grasping/grasp_markers` — маркеры для RViz,
  - `/camera/image_display` — картинка камеры.

Если объект в кадре не найден, используется априорная поза блока (параметры `block_x`, `block_y`). Это видно в логе: `... у блока (0.350, -0.750)`.

```
ros2 topic echo /xarm_grasping/grasp_candidates --once
```

---

12. Проверка, что всё работает

После `ros2 launch cart_xarm_bringup cart_xarm6_gz.launch.py` подождите ~15 с и выполните во втором терминале:

```
ros2 control list_controllers
  # joint_state_broadcaster ... active
  # xarm6_traj_controller   ... active

  ros2 topic echo --once --field position /joint_states
  # ≈ [1.5708, -0.9, -1.3, 0.0, 1.0, 0.0] — рука в начальной позе

  ros2 topic hz /camera/depth/points          # ~20–30 Гц
  ros2 run tf2_ros tf2_echo base_link drive_wheel_left_link   # TF колёс есть
  ros2 run tf2_ros tf2_echo base_link link_base               # z ≈ 0.849 — центр платф
  ормы
  ros2 topic echo /xarm_grasping/grasp_candidates --once
```

Дерево TF целиком:

```
ros2 run tf2_tools view_frames   # создаст frames_*.pdf
```

---

13. Типичные проблемы

Симптом	Причина и решение
`Package 'cart_xarm_bringup' not found`	не выполнен `source ~/ros2k_ws/install/setup.bash` в этом терминале
`ros2: invalid choice: 'control'`	не установлен `ros-kilted-ros2controlcli`
`Another world of the same name is running`	от прошлого запуска осталась gz sim: `pkill -f "gz sim"`
Gazebo падает с `Scene already exists with name: scene`	Sensors-система подключена дважды: она должна быть только в world-файле, модель собирается с `model_sensors_plugin:=false` (уже так в `cart_with_arm.urdf.xacro`)
Сценарий A: Gazebo сразу закрывается, `Unable to find uri[...]`	нет интернета для моделей Fuel при первом запуске
RViz: `No transform from [drive_wheel_*]`	не запущен `joint_state_merger` или нет моста `/cart/wheel_joint_states`; проверьте `ros2 topic echo /cart/wheel_joint_states`
Рука в RViz не в той позе, что в Gazebo	`robot_state_publisher` читает `/merger/joint_states` (рука + колёса); проверьте `ros2 topic echo --once /merger/joint_states` — там должны быть углы руки, а не нули
MoveIt: `Robot model not loaded`	ошибка разбора URDF MoveIt, смотрите первые строки лога `move_group`
MoveIt: `PATH_TOLERANCE_VIOLATED`	рука упирается в препятствие в Gazebo (MoveIt его не видит); смените начальную позу или цель
`grasp_analyzer`: «Облако точек ещё не получено»	камера ещё не начала публиковать; если сообщение не пропадает, проверьте `ros2 topic hz /camera/depth/points`
Traceback / `exit code -11` при Ctrl+C	ноды на Python и rviz2 так завершаются по сигналу; это не ошибка

14. Структура репозитория

```
src/
  ├── r2_robot/
  │   ├── quasi_cart_description/   # URDF тележки; cart_with_arm.urdf.xacro + cart-ar
  m.xacro (крепление руки)
  │   └── cart_xarm_bringup/        # сценарий B: launch (gz, rviz), миры cart_table*,
  gz_bridge.yaml, rviz
  ├── xarm_gz_bringup/              # сценарий A: рука на столе в Gazebo (table_gz.worl
  d)
  ├── xarm_grasping/                # grasp_analyzer.py + grasp_generator.py
  └── xarm_ros2/                    # upstream UFactory (description, controller, movei
  t_config, ...),
                                    # с локальными правками под Kilted
```

Ключевые файлы:

Файл	Что внутри
`cart_xarm_bringup/launch/cart_xarm6_gz.launch.py`	запуск тележки с рукой в Gazebo (+MoveIt)
`cart_xarm_bringup/rviz/cart_xarm_moveit.rviz`	RViz MoveIt: тележка + MotionPlanning + маркеры + камера
`cart_xarm_bringup/launch/cart_xarm6_rviz.launch.py`	запуск только в RViz
`cart_xarm_bringup/config/gz_bridge.yaml`	мосты Gazebo ↔ ROS: clock, камера, колёса, cmd_vel, odom
`cart_xarm_bringup/worlds/cart_table_offline.world`	мир (системные плагины gz, стол, блок)
`quasi_cart_description/urdf/cart_with_arm.urdf.xacro`	верхний URDF тележки с рукой
`xarm_description/urdf/xarm6/xarm6.ros2_control.xacro`	интерфейсы ros2_control руки + начальная поза
`xarm_moveit_config/launch/_robot_moveit_common.launch.py`	move_group + RViz MoveIt (аргументы `use_sim_time`, `publish_attach_tf`, `rviz_config`)